

Notes about this project

For implementation details of the algorithms in the project, see [README.md](#)

This is just a practice project for the course in Backend Programming at [Gokstad Akademiet](#), year 2, semester 1.

Its purpose is practicing search algorithms.

Overview

- [Search Algorithms](#)
 - [BFS - Breadth-First Search](#)
 - [Dijkstra's Algorithm](#)
 - [Bellman-Ford Algorithm](#)
 - [DFS - Depth-First Search](#)
-

Search Algorithms

- [LinearSearch\(\)](#)
 - [BinarySearch\(\)](#)
 - [Bfs\(\)](#)
 - [Dfs\(\)](#)
-

BFS - Breadth-First Search

- Waterdrop analogy
- Ripples expand outward
- FIFO
- Level by level / one hop at a time
- Queues and HashSets 

Pros:

- Shortest path guarantee - in an unweighted graph, when BFS hits target node, it is mathematically guaranteed to be the shortest path (fewest edges traveled)
- Optimal for close targets - if the target is close to the start, BFS will find it almost instantly. No extra exploration.
- Iterative Safety (uses a loop and a queue object), does not risk crashing the call stack

Cons:

- Memory-heavy on wide graphs - i.e., social network. Queue has to hold an entire layer of the graph at once. Space complexity is $O(W)$ (maximum width), which can balloon to $O(V)$.

- Fails on weighted graphs if edges have varying costs → upgrade to Dijkstra's Algorithm (which swaps the FIFO Queue for a priority queue.)
- No path depth. - It doesn't naturally track deep pathways or cycles as elegantly as DFS.

Real-World Use Cases:

- Social Network Degrees of Separation: Finding if someone is a "2nd degree" or "3rd degree" connection on LinkedIn, or building a friends-of-friends recommendation list.
- Web Crawlers: Google originally utilized BFS to index the prominent links on a homepage before crawling into deep, obscure sub-pages.
- Peer-to-Peer (P2P) Networks: Protocols like BitTorrent use BFS to locate neighboring peer nodes within 1 hop, then 2, etc.
- State-Space Search(Games): Mapping out every possible sequence of moves in a puzzle or game, level-by-level to find the fastest winning state.

Edge Cases & Debugging Traps:

How the code fails.

- The Infinite Loop Trap:
 - The danger: If you forget to use a `visited HashSet` (or fail to mark a node as visited immediately upon enqueueing) and your graph contains a `cycle` (e.g., node A points to node B, node B points back to node A), the queue will bounce them back and forth forever.
 - The fix: Always check `!visited.Contains(neighbor)` and add it to the `visited` set the exact moment you enqueue it.
- The `KeyNotFoundException` Crash:
 - The danger: When retrieving a node's neighbors, your code might look up a node in your dictionary that hasn't been officially added as a key.
 - The fix: Use `.ContainsKey()` or `TryGetValue` before attempting to iterate through a node's neighbor list.
- Disconnected Graphs:
 - The danger: If the target node lies in an isolated, disconnected component of the graph, your BFS will exhaust the queue and return without finding it - failing silently without your knowledge.
 - The fix: Ensure your pathfinding return logic cleanly handles an empty queue state by returning an empty result or path-not-found sentinel value like `null` or an empty list.

Dijkstra's Algorithm

The Bridge from BFS to Dijkstra's.

- The Unweighted VS Weighted Difference:
 - In `BFS`, path length is measured strictly by the number of edges (hops). BFS assumes all connections are equal.
 - In `Dijkstra's`, path length is measured by the sum of weights along those edges (such as minutes of traffic, toll costs, or physical distance)
- The Family Connection:
 - `Dijkstra's` is a generalized sibling of `BFS`. If you replace every weighted edge in a graph with a chain of unweighted, single-unit placeholder nodes, running `Dijkstra's` on the original graph

yields the exact same steps as running **BFS**. If all edge weights are identical, **Dijkstra's** behaves exactly like **BFS**.

Core Metaphor - The Cost Conscious Explorer

You are exploring a weighted map. Instead of walking in standard, rigid concentric rings, you carry a stopwatch. You explore roads strictly in order of their total travel time from the start, always choosing the shortest unvisited path next. This greedy, cost-prioritized approach guarantees that when you finally step onto a destination, you have taken the absolute most efficient route.

The One Big Data Structure Swap

Dijkstra's code structure is nearly identical to BFS, save for the engine that holds the active nodes.

- **BFS** uses a **Queue (FIFO)** to explore level-by-level.
- **Dijkstra's** swaps the queue for a **Priority Queue** (typically a **Binary Heap**). Instead of grabbing the oldest node, it dynamically sorts the nodes and always pops the node with the lowest accumulated cost.
- The Shared Invariant:
 - At the **i-th** iteration, both **BFS** and **Dijkstra's** have already popped the **i-th** closest nodes to the start node of their respective queues. They only differ in how they calculate "closeness" (hops VS accumulated weight).

Step-By-Step Mechanics

1. Initialize: set the starting node's distance to **0** and all other nodes to **Infinity**.
2. Select: grab the unvisited node with the smallest known accumulated distance.
3. Relax Neighbors (Core Loop): for each neighbor of your current node, calculate the total distance to reach it *through* your current node.
4. Update: if this newly calculated distance is cheaper than the neighbor's previously recorded distance, update it and record the current node as its "parent" (for path backtracking later).
5. Finalize: mark the current node as fully processed/visited. Repeat until all reachable nodes are processed.

Pros and Cons

Pros:

- Optimal Path Guarantee - Always guarantees finding the shortest path on weighted network with non-negative edge weights.
- Highly Efficient - When optimized using a Binary Heap, runs in highly practical $O((V + E) \log V)$ time complexity.

Cons:

- Negative Edge Vulnerability - fails on graphs with negative edge weights. Assumes that once a node is "visited", shortest path is locked. (You must use **Bellman-Ford Algorithm** to handle negative weights!)

Real-World Use Cases

- GPS navigation (maps, car nav)
- Network packet routing protocols

- Game AI navigation
- Robot path planning

Edge Cases

- Disconnected Graph:
 - If a node is completely unreachable, its cost remains **Infinity**. Always include a guard check → if the cheapest node popped off the priority queue has a distance of **Infinity** -> stop searching.
 - Large-Scale Performance Bottlenecks:
 - On dense networks, sorting a basic array to find the lowest-cost node slows **Dijkstra's** down to $O(V^2)$. In production systems, always use a **Priority Queue** to maintain logarithmic speed.
-

Bellman-Ford Algorithm

"Brute-Force Relaxation" (😓)

While **Dijkstra's** is **greedy**, **Bellman-Ford** is robustly thorough.

- Instead of systematically picking nodes, it looks at **every single edge** in the graph and tries to "relax" it (checking if it can find a shorter path to the destination node through the source node).
- It repeats this massive scan of all edges exactly "**V - 1** times" (where **V** is the number of vertices/nodes in the graph).

The Math Logic

The longest possible simple path (without loops) connecting any two nodes in a graph with **V** vertices can have at most

V - 1 edges. Therefore, relaxing all edges **V - 1** times mathematically guarantees that the shortest path to all nodes are calculated.

The Superpower: Detecting Negative Cycles

If the graph contains **negative cycles** (a loop whose weights sum to a negative number), an algorithm could loop infinitely to achieve a path cost of negative infinity.

Dijkstra's will get trapped or calculate incorrect distances.

Bellman-Ford catches this easily:

- After relaxing all edges **V - 1** times, it runs one extra relaxation pass (the **V-th** pass).
- If any distance can *still* be shortened on this extra pass, it means there is a **negative cycle** spinning somewhere in the graph. The algorithm stops and sounds the alarm (returns **false** or throws an error).

Code Implementation Template

(added before we learned anything about this during lectures)

Standard **Bellman-Ford Algorithm** setup using a simple list of edges (easiest way to represent graphs for this algorithm):

```
using System;
using System.Collections.Generic;
```

```
namespace AlgorithmPractice.Pathfinding;

internal class BellmanFord
{
    // A simple representation of a weighted edge
    internal struct Edge
    {
        public int Source;
        public int Destination;
        public int Weight;

        public Edge(int source, int destination, int weight)
        {
            Source = source;
            Destination = destination;
            Weight = weight;
        }
    }

    internal static bool RunBellmanFord(List<Edge> edges, int vertexCount, int
startNode, out int[] distances)
    {
        distances = new int[vertexCount];

        // initialize all as "Infinity" (int.MaxValue)
        Array.Fill(distances, int.MaxValue);
        distances[startNode] = 0; // distance to ourselves is always 0

        // relax all edges "V" - 1 times
        for (int i = 1; i < vertexCount; i++)
        {
            foreach (var edge in edges)
            {
                // only relax if we already found valid path to source node
                if (distances[edge.Source] != int.MaxValue &&
                    distances[edge.Source] + edge.Weight <
distances[edge.Destination])
                {
                    distances[edge.Destination] = distances[edge.Source] +
edge.Weight;
                }
            }
        }
        // run the V-th pass to check for negative-weight cycles
        foreach (var edge in edges)
        {
            if (distances[edge.Source] != int.MaxValue &&
                distances[edge.Source] + edge.Weight <
distances[edge.Destination])
            {
                // if we can still relax an edge, a negative cycle exists
                Console.WriteLine("Warning: Graph contains a negative-weight
cycle!");
                return false;
            }
        }
    }
}
```

```
    }  
    return true; // execution successful, no negative cycles  
  }  
}
```

Summary of Dijkstra's VS Bellman-Ford

- Dijkstra's:
 - Time Complexity: $O((V + E) \log V)$. Using a priority queue/binary heap.
 - Handling Negative Weights: No (fails completely).
- Bellman-Ford:
 - Time Complexity: $O(V * E)$. Slower because it iterates over all edges repeatedly.
 - Handling Negative Weights: Yes (and detects negative cycles safely).

DFS - Depth-First Search

The Maze Runner.

Imagine a physical labyrinth:

You place your hand on the right-hand wall and walk forward, taking every turn you can, and keep diving deeper and deeper down a single path until you hit a dead end.

Then you backtrack to the last intersection, and dive down the next unexplored branch.

The Native Call Stack Advantage

Unlike **BFS** (naturally iterative, explicit queue), **DFS** is most commonly implemented using **recursion**.

- How it works:
 - Instead of managing a manual stack object, a recursive DFS elegantly utilizes the **CPU's native stack call** to remember where it came from. Each recursive function call is pushed onto the stack, and when a branch hits a dead end, the function returns, which automatically backtracks to the previous caller.
- The alternative:
 - You can write DFS iteratively, but you must manually use a **Stack** (LIFO, Last-In, First-Out) data structure.

Complexity and Performance

- Time Complexity: $O(V + E)$. Like BFS, DFS must visit every vertex (V) and check every edge (E) in the connected component.
- Space Complexity: $O(H)$ where **H** is the maximum depth/height of the graph.
 - Worst case: if graph is single long line, degrades to $O(V)$ because every single node's recursive call sits on the stack at the same time.
 - Advantage over BFS: In a very wide graph with a massive branching factor, DFS is highly memory-efficient compared to BFS. BFS must store an entire "wave" of nodes in its queue, whereas DFS only needs to store the single active path from the root to the current node.

Pros and Cons

Pros:

- Highly Memory-Efficient on Wide Graphs - requires much less memory than BFS when the tree or graph spreads out quickly
- Perfect for Backtracking - ideal for exploring all possible paths, configurations, or solutions to a puzzle.
- Cycle & Cycle Detection - uniquely suited to finding "back-edges" (edges pointing back to an ancestor), making it the gold standard for cycle detection.

Cons:

- No Shortest Path Guarantee - will aggressively follow a convoluted, massive path to a target node even if there is a short, direct path right next to the starting point.
- Vulnerable to Deep Graphs - if graph is exceptionally deep, recursion can easily exceed the CPU's stack limits and cause a crash.
- Recursion Overhead - recursive implementations carry a bit of call-stack overhead compared to straightforward loops

Real-World Use Cases

- Cycle Detection - detecting cycles in network routers to prevent loops, identifying deadlocks in operating systems.
- Topological Sorting - used by build systems (like npm, NuGet, Webpack) to linearly order and execute tasks with complex dependency chains.
- Solving Puzzles and Mazes - the core algorithm behind solving Sudoku, the N-Queens problem, or auto-generating physical mazes.
- Strongly Connected Components - algorithms like **Tarjan's** or **Kosaraju's** use DFS to identify groups of nodes that are completely interconnected.

Edge-Cases and Debugging Traps

- The Infinite Cycle Trap:
 - The danger: if your graph contains a cycle, and you forget to track visited nodes, your recursive calls will loop infinitely until your program crashes.
 - The Fix: Always pass you **visited** HashSet into your recursive method, and check **!visited.Contains(neighbor)** before making the next recursive call.
- **StackOverflowException** Crash:
 - The danger: if the recursion goes too deep (infinite loop, graph too deep), the CPU's call stack runs out of allocated memory.
 - The Fix: if your database holds incredibly deep paths, consider writing an **iterative DFS** using an explicit **Stack** object on the heap, or use **Iterative Deepening DFS (IDDFS)** which limits depth dynamically.